

csiro-biomass

8位Solution | PixelScale

Rank	8
Team	PixelScale
Author	Paritosh Kumar Tripathi
Topic ID	671363
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/671363>

Retrieved with Kaggle CLI on 2026-07-18.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

1. 概要と重要な洞察

私たちのsolutionは、すべてDINOv3 Huge backboneを基盤とする3つの異なるpipelineのweighted ensembleです。

最終提出は次を組み合わせました。

1. **SSA適応pipeline (50%)** : subspace alignmentを使うTest-Time Adaptation。
2. **Species-aware pipeline (30%)** : species metadataを2段階で注入。
3. **Dual-backbone pipeline (20%)** : DINOv3とSigLIP featureを融合。

2. Core backbone (Takumiの担当)

- 私はscaling lawの傾向を強く信じ、計算資源の許す範囲で最大のmodelを使うことを目指しました。
- dataが限られるコンペでは、大規模dataでpretrainされた強いbackboneの活用が重要だという洞察を得ました。
- コンペの大半をDINOv3のtraining recipe最適化へ費やしました。同じarchitectureを使う他solutionに対する優位性はここにあったと思います。
- model sizeを増やすとPublic LBが大きく伸びました。画像解像度を増やすよりmodel sizeを増やす方が効果的でした。A100のmemory制約から、多くのpipelineを次へ統一しました。
- **Model:** vit_huge_plus_patch16_dinov3.lvd1689m
- **Image size:** 768px

backboneを変えただけでどれほど効いたかを強調したいと思います。

- convnext_tiny → vit_base_patch16_dinov3.lvd1689m: 0.61 → 0.67
- vit_base_patch16_dinov3.lvd1689m → vit_large_patch16_dinov3.lvd1689m: 0.69 → 0.72
- vit_large_patch16_dinov3.lvd1689m → vit_huge_plus_patch16_dinov3.lvd1689m: 0.72 → 0.74

Head architecture

SiLU activationとdropoutを使う標準的なregression headです。

```
nn.Sequential(  
    nn.Linear(in_features, hidden),  
    nn.SiLU(),  
    nn.Dropout(0.3),  
    nn.Linear(hidden, out_features),  
)
```

Training strategy

少量dataで巨大modelの学習を安定させるため、AdamW と Mixed Precision (AMP) を使う2段階学習を行いました。

- **Loss function:** `nn.SmoothL1Loss()`
- **Batch size:** 1で `grad_accum_steps=8` とし、effective batch sizeを8に設定。

Stage 1: Headのみ (warmup)

- **目的:** transformerをfine-tuneする前にweightを安定させる。
- **Epoch:** 15
- **LR:** `3e-4`、`ReduceLR0nPlateau`。

Stage 2: Backbone全体をfine-tuning

- **目的:** overfittingを避けつつHuge backboneを適応させる。
- **Epoch:** 35
- **Weight decay:** `0.2` (強いdecayがregularizationに不可欠だった)。
- **Scheduler:** 厳格なwarmup付きcosine。

```
# Stage 2 Scheduler Configuration
CosineLRScheduler(
    lr_min=1e-6,          # Target minimum LR
    warmup_t=3,           # Crucial 3-epoch warmup
    warmup_lr_init=1e-7,  # Start near zero
    cycle_limit=1,        # Single cycle (no restarts)
    t_in_epochs=True
)
```

Data augmentation

lighting variationへ対応し、invariant featureを学ばせるため、特に `RandomShadow` と `RandomGamma` を含む Albumentationsを使いました。

```
A.Compose([
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.RandomRotate90(p=0.5),
    A.ColorJitter(brightness=0.2, contrast=0.1, saturation=0.2, p=0.5),
    A.RandomGamma(gamma_limit=(80, 120), p=0.3),
    A.RandomShadow(num_shadows_limit=(1, 3), shadow_dimension=5, p=0.5),
    A.Resize(self.img_size, self.img_size),
    A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])
```

Validation strategy

当初は `sampling_date` を使う **GroupKFold** を試しましたが、fold間の分布分散が大きくなり、一部のmodelが十分なdata variationを学べませんでした。Public LBも改善しませんでした。

そこでCVとPublic LBの不一致を受け入れ、binとspeciesに基づくstratified splitでCVを最大化しました。

```
# Create a stratification key combining total biomass bin and species
df['strat_key'] = df['total_bin'].astype(int) * 100 + df['species_id'].astype(int)

skf = StratifiedKFold(
    n_splits=self.config.n_folds,
    shuffle=True,
    random_state=self.config.random_state
)
```

Species-aware prediction

推論時にmetadataが得られない、または信頼できない場合にもspecies metadataを有効利用するため、次の2段階を使いました。

1. **Species model:** 同じhead構成の `dinov3_large` を画像からspeciesを分類するよう学習。
2. **Inference:** 各speciesへ属する確率をSpecies modelで予測し、その確率をmain Biomass Prediction modelの入力featureにする。

3. 重要な最適化（Diptyajit Dasの担当）

backboneがpowerを提供する一方、Diptyajitによるtargetとresolutionの実験が、Private LBで必要な安定性と一般化性能をもたらしました。

A. Targetの再定式化

標準targetを直接予測せず、生物学的componentにより整合するようregression問題を再定式化しました。

- **旧target:** `Dry_Green_g`、`GDM_g`、`Dry_Total_g`
- **新target:** `Dry_Clover_g`、`GDM_g`、`Dry_Total_g`
- **効果:** DINOv3 Huge variantで特に重要で、Public LBが**0.73→0.74**へ改善。

B. Auxiliary headと安定性

Stage 1でspecies classificationとともにNDVI regression auxiliary headを加えました。`vit_large` は0.73へ改善しましたが、`vit_huge_plus` のscoreにはほぼ影響しませんでした。

C. 「Golden」 resolution (640px)

768pxが**最良Public LB**だった一方、640pxは一般化性能が高く、**single model最高Private LB 0.657**になりました。ただしPublic LBが0.75と低かったため、最終評価用には選ばませんでした。

4. Pipeline A: SSA AdapterとMLP head（Paritoshの担当）

Ensemble weight: 0.5

このpipelineは、高価な再学習をせずtrain/testのdomain shiftへ対応しました。標準fine-tuningの代わりに、軽量のTest-Time Adaptationとして**Significant Subspace Alignment (SSA)** を使いました。

「2-layer」 MLP head

- 標準linear headを深いprojection headへ置き換え、Public LBを**0.74→0.76 (+0.02)**、Private LBを****0.635→0.646 (+0.01)****へ改善しました。
- 最終的には2-layerより僅かに良かった3-layer MLP headを採用しました。

```
nn.Sequential(  
    nn.Linear(input_dim, 512), # Hidden Layer 1  
    nn.BatchNorm1d(512),  
    nn.ReLU(),  
    nn.Dropout(0.3),  
  
    nn.Linear(512, 256),      # Hidden Layer 2  
    nn.BatchNorm1d(256),  
    nn.ReLU(),  
    nn.Dropout(0.2),  
  
    nn.Linear(256, 128),      # Hidden Layer 3  
    nn.BatchNorm1d(128),  
    nn.ReLU(),  
    nn.Dropout(0.2),  
  
    nn.Linear(128, n_targets)  
)
```

SSA Adapter（Test-Time Adaptation）

backboneとheadの間へlearnable linear adapterを挿入しました。推論時はbackboneをfreezeし、このadapterだけを更新して、事前計算したPCA統計量 μ_s と U を使いtest画像featureをtrain分布へ合わせました。詳細：
<https://arxiv.org/pdf/2410.03263>

Public LBはほぼ変わりませんでした。この方法を信頼したところPrivate LBが**0.646→0.652**へ改善しました。

5. Pipeline B: Species-aware injection（Takumiの担当）

Ensemble weight: 0.3

1. **Stage 1（Species model）** : vit_large_patch16_dinov3 で15 speciesを分類。
2. **Stage 2（Biomass model）** :
 - Backbone: vit_huge_plus_patch16_dinov3
 - Input: image feature + 予測したSpecies確率を連結
 - Target: Dry_Clover_g と GDM_g を予測してtotalを再構成

6. Pipeline C: SigLIPとのdual-backbone fusion（Takumiの担当）

Ensemble weight: 0.2

異なるsemantic granularityを捉えるため、2つの最先端vision modelのfeatureを融合しました。

- **Branch 1:** DINOv3 Huge（768px） — structureとlocal detailに優れる。
- **Branch 2:** SigLIP vit_so400m_patch16_siglip_512（512px） — semantic understandingに優れる。
- **Fusion:** 両backboneのfeatureを連結してMLP headへ入力。

```
# Feature Fusion Logic
feat_dino = dino_backbone(img_768) # DINOv3 Features
feat_siglip = siglip_backbone(img_512) # SigLIP Features

# Concatenate: [Backbone_L, Backbone_R, SigLIP_L, SigLIP_R]
combined_features = torch.cat([feat_dino, feat_siglip], dim=1)
```

7. 最終ensembleと結果

最終提出は3 pipelineのweighted averageです。SSA modelを0.5と大きくしたのは、private test setの distribution shiftに対するrobustnessを重視したためです。最終ensembleで最後の小数点以下を伸ばせました。

```
# Final Ensemble Logic
final_submission = (
    0.5 * prediction_ssa + # Pipeline A (Robustness)
    0.3 * prediction_species + # Pipeline B (Metadata Context)
    0.2 * prediction_siglip # Pipeline C (Feature Diversity)
)
```

Validation strategy

total_biomass_bin * species_id のcomposite keyでstratifyする**StratifiedKFold（5 fold）**を使い、各foldにspeciesとweight classの代表的な分布が含まれるようにしました。

Performance

- **Public LB:** 0.760
- **Private LB:** 0.652

補足Q&A

Target再定式化はすべてのmodelで有効だったか

minghigh（質問）: Dry_Green_g、GDM_g、Dry_Total_g より、Dry_Clover_g、GDM_g、Dry_Total_g を使う方が良かった、という意味でしょうか。すべてのmodelで同じでしたか。

Paritosh Kumar Tripathi (著者) : すべてのmodelでは試していませんが、DINOv3 HugeではCVとLBの両方が確かに改善しました。diversityを重視したため、私のMLP head (Pipeline A) は5targetすべてを予測し、Pipeline BとCは3targetを予測してそこから5targetを再構成しました。